

ReLU Town Hall: Unsupervised learning

Clustering and dimensionality reduction

Gian Marco Miccio Connor O'Reilly

ReLU NTNU

Thursday 23rd October, 2025



Goal: understand learning without labels, what PCA is, and what to do if it fails.

Unsupervised learning

- What, when, and why?

Clustering

- What are clusters?
- k-means clustering

Dimensionality reduction

- Why?
- Principal component analysis

Limitations of linear methods

- PCA on non-linear data
- Kernel PCA

Non-linear dimensionality reduction

- A paradigm shift
- t-SNE
- UMAP
- PaCMAP

(Please ask questions at any point!)



Unsupervised learning

What, when, and why?

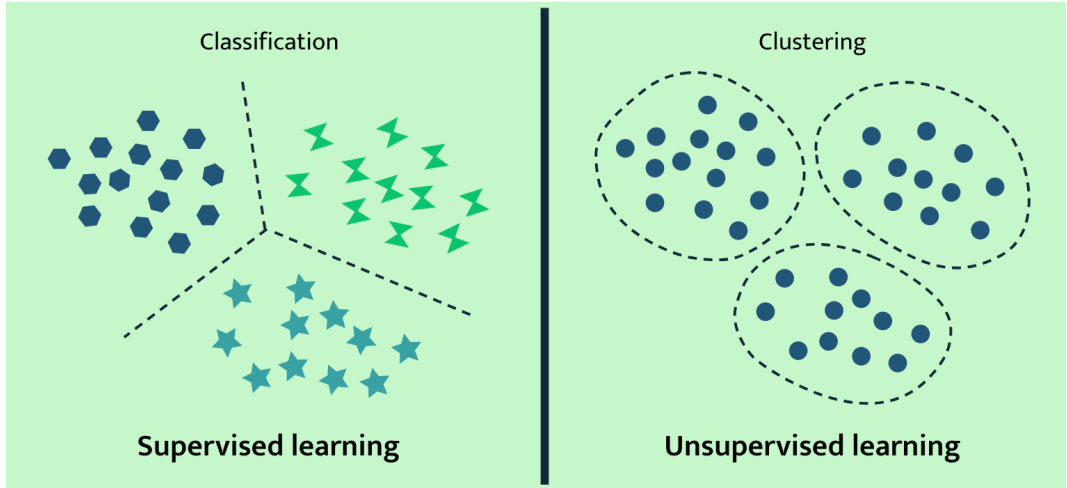


a little reminder on the terms we are using

- **Supervised Learning:** when we know the true answer for the response in the dataset, meaning we know which class a training data belongs to
- **Unsupervised Learning:** this would be the case in which the classes we are trying to assign are created by us rather than being something that's already been studied about the dataset



Unsupervised learning



There are three main tasks that unsupervised learning can carry out

- 1 **Clustering:** group similar data together so that observations in the same cluster are more similar than observations in other clusters
- 2 **Dimensionality Reduction:** reduce the number of features while maintaining as much important information as possible
- 3 **Association Rule:** discover strong relationships or correlations between variables in a large dataset



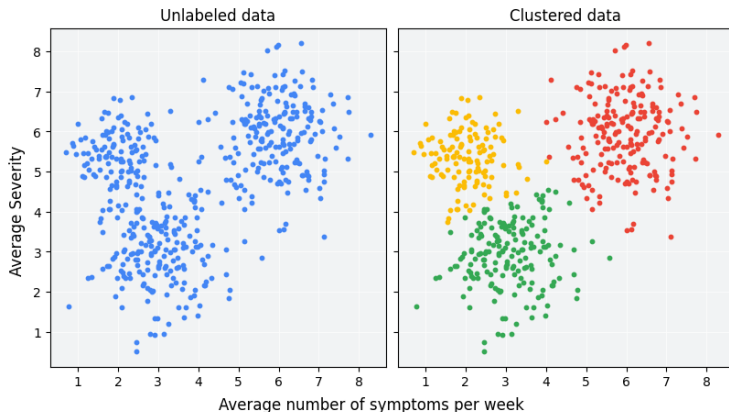
Clustering

What are clusters?



Clusters

definition: a cluster is a collection of points that are **similar** to each other and **dissimilar** to other points in other groups



Clustering

k-means clustering



k-means clustering

To calculate clusters, the most commonly used algorithm (and also the most intuitive) is the **k-means clustering** algorithm, and it consists of an initialization plus two iterative steps:

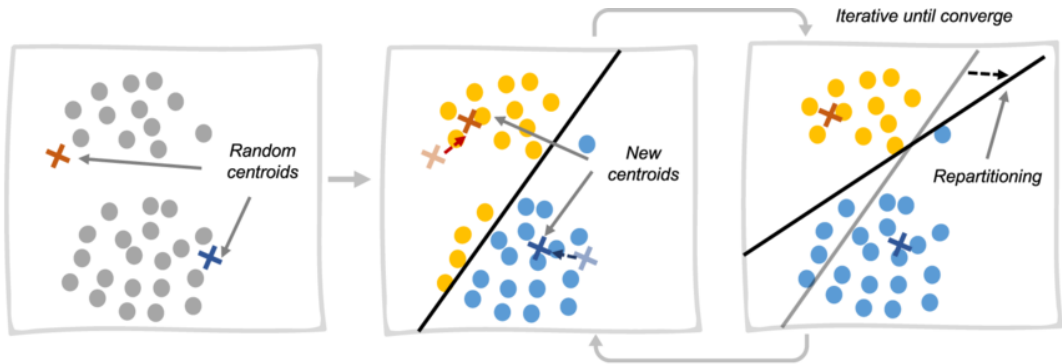
Initialize two random centroids (i.e. the mean) and assign the points to the closest one

- 1 calculate the centroid of the points of a cluster, aka the "center"
- 2 reassign all the points to the closest centroid

Algorithm ends when there are no changes any more!



k-means clustering



Input: distance matrix D & number of clusters k



k-means clustering

let's define this mathematically (so I can be happy and to show you math is not as scary as it looks like)



k-means clustering

The k-means algorithm aims at minimizing this **loss function**

$$L(\mathbf{S}, \boldsymbol{\mu}) = \sum_{j=1}^K \sum_{\mathbf{x}_i \in S_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

which is the distance of every point x_i from the **centroid** of its cluster S_j to which it's assigned

$$\boldsymbol{\mu}_j = \frac{1}{n_j} \sum_{\mathbf{x}_i \in S_j} \mathbf{x}_i$$



The k-means algorithm

The k-means algorithm proceeds as follows:

- **Initialization:** Randomly initialize the K centroids $\mu_1^{(0)}, \mu_2^{(0)}, \dots, \mu_K^{(0)}$
- **Iterative steps:** Repeat until convergence:
 - 1 **Assignment step:** Assign each data point $\mathbf{x}_i$ to the cluster with the closest centroid

$$S_j^{(t)} = \left\{ \mathbf{x}_i : \|\mathbf{x}_i - \mu_j^{(t)}\|^2 \leq \|\mathbf{x}_i - \mu_k^{(t)}\|^2 \forall k \right\}$$

- 2 **Update step:** Recompute the centroids based on the new cluster assignments

$$\mu_j^{(t+1)} = \frac{1}{n_j^{(t)}} \sum_{\mathbf{x}_i \in S_j^{(t)}} \mathbf{x}_i$$

The algorithm stops when the assignments no longer change or the loss $L(\mathbf{S}, \mu)$ converges



k-means clustering

Let's take a look at a demo to see the algorithm in action in [this interactive k-means demo](#).



k-means clustering

Some things to keep in mind for this algorithm:

- it's very **simple**, but that means that sometimes it's ineffective
- it's guaranteed to converge to a **local minimum**, but not a global one necessarily
- it's **dependent on initialization**, as different initial centroids can lead to different results
- scaling matters, as typically you would want to standardize your data



k-means clustering

How can you tell which number of k to pick?

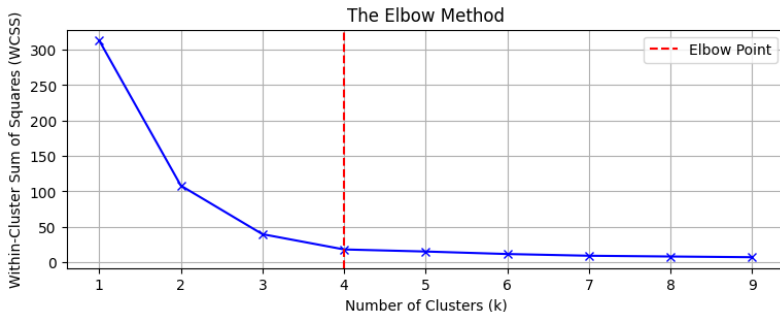
- if k is **low**, for example 2, you might miss a lot of subgroups, and the two groups might separate where you don't want to
- if k is too **high**, for example close to the number of total points, then the clustering loses meaning

Note: sometimes you might simply want a specific k for external reasons (makes sense for your project, empirically you know it to be true etc)



k-means clustering

For this there's the **elbow rule**, when loss function at the final step of the algorithm stops improving significantly



- Loss function and centroid can be changed! You could for example use the Manhattan distance (L^1) with median rather than mean
- **Assumptions and limitations**
 - Works best for spherical, equally sized clusters with similar variance.
 - Sensitive to outliers (since it uses squared distances).
 - Requires continuous, numerical features — not ideal for categorical data.



Dimensionality reduction

Why?



Dimensionality Reduction

It's the process of reducing the number of random variables (or features) while still maintaining the most amount of important information possible

- improves **speed and performance** of a model (you're getting rid of superfluous information)
- allows for better **data visualization**, it can let you plot things in 2D or 3D



Dimensionality Reduction

You can perform either

- 1 **feature selection** where you keep a subset of the original features (good for explainability)
- 2 **feature extraction** where you combine and transform feature to concentrate the important information (can lose interpretability)



Let's do a little thought exercise to understand a concept called
Curse of Dimensionality

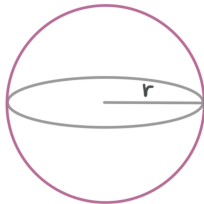


Curse of Dimensionality

Idea: Imagine you have a space, any space, a universe, in the shape of a sphere of radius 1

For this thought experiment imagine that this can be done with any number of dimensions, 1D, 2D, 3D but even higher! It can be any space $\mathbb{R}^d$

Question: how far along the radius do we need to go capture 10% of the total space?



Curse of Dimensionality

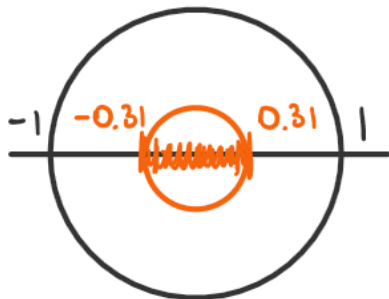
For a **line** ($d=1$) this is fairly straightforward: $r = 0.1$ is the answer



Curse of Dimensionality

For a **circle** (d=2) you divide the portion of area covered by a circle of radius r vs the total area, which is a circle of radius 1

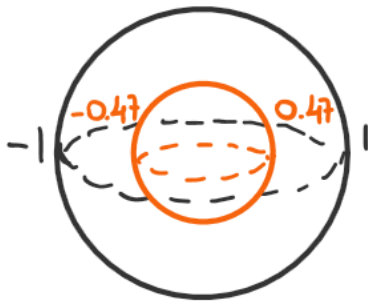
$$\frac{\pi r^2}{\pi 1^2} = 0.1 \quad \Rightarrow \quad r = \sqrt{0.1} \approx 0.31$$



Curse of Dimensionality

For a **sphere** (d=3) you divide the portion of volume covered by a circle of radius r vs the total volume with radius 1

$$\frac{\frac{4}{3}\pi r^3}{\frac{4}{3}\pi 1^3} = 0.1 \quad \Rightarrow \quad r = \sqrt[3]{0.1} \approx 0.47$$



Curse of Dimensionality

This can be carried out in any dimension d ! and for a universe of **dimension $d=100$** the same formula applies

$$r = \sqrt[100]{0.1} = 0.1^{\frac{1}{100}} \approx 0.97$$

In other words, you need to walk 97% of the radius in to cover just 10% of the space. So if you have 100 features or 100 covariates or 100 variables you need to see 97% of their extension to see 10% of the total!



Curse of Dimensionality

Back in the Statistics and Linear Regression Bootcamp we went on a whole journey about the **Covariance Matrix**

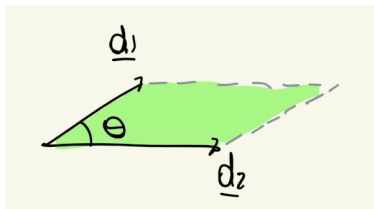
$$S = \begin{bmatrix} s_{11} & s_{12} \\ s_{21} & s_{22} \end{bmatrix} = \frac{1}{n-1} \begin{bmatrix} d_1' d_1 & d_1' d_2 \\ d_2' d_1 & d_2' d_2 \end{bmatrix} \quad \text{where} \quad \underline{d}_1 = \underline{c}_1 - \bar{x}_1 \underline{1} = \begin{pmatrix} x_{11} - \bar{x}_1 \\ x_{21} - \bar{x}_1 \\ \vdots \\ x_{n1} - \bar{x}_1 \end{pmatrix}$$

And we showed that to measure variance for a whole dataset (the **generalized variance**) we could look at the **determinant** of the covariate matrix $\det S$



Curse of Dimensionality

Back in the Linear Algebra Bootcamp we mentioned that the determinant can be interpreted as a volume (or area in a 2D space) delimited by vectors



Idea: Curse of dimensionality applies to Variance as well!

If we have a lot of covariates, to capture just a little bit of variability we need to cover a lot of the spectrum of how far our covariates can extend



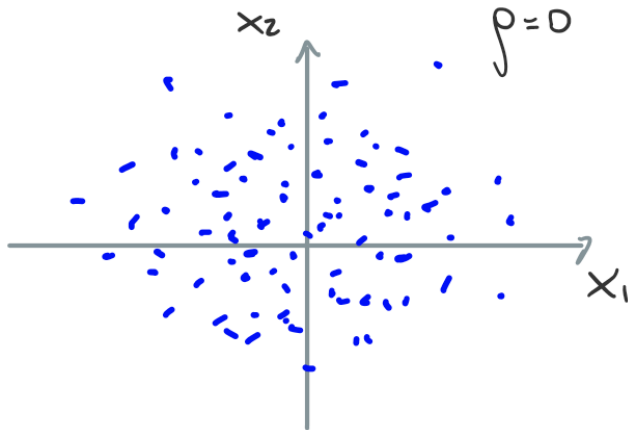
Dimensionality reduction

Principal component analysis



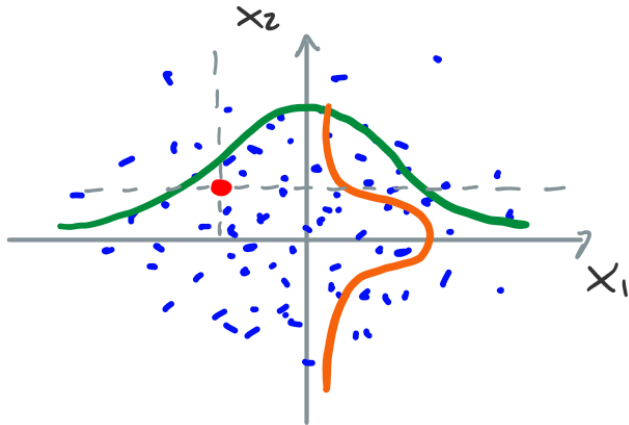
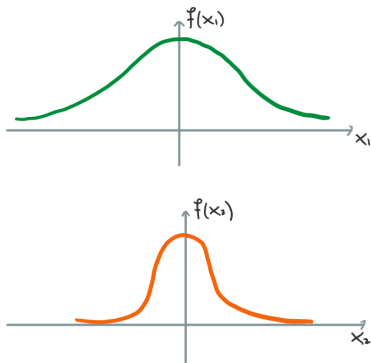
PCA

Before we do any math, I will walk you through some graphs, and the first one is a dataset with two **independent** features



PCA

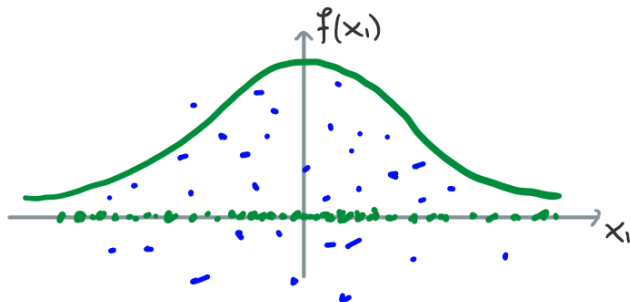
You can think of both features having their own distribution, and each data point is a realization of both these distributions



PCA

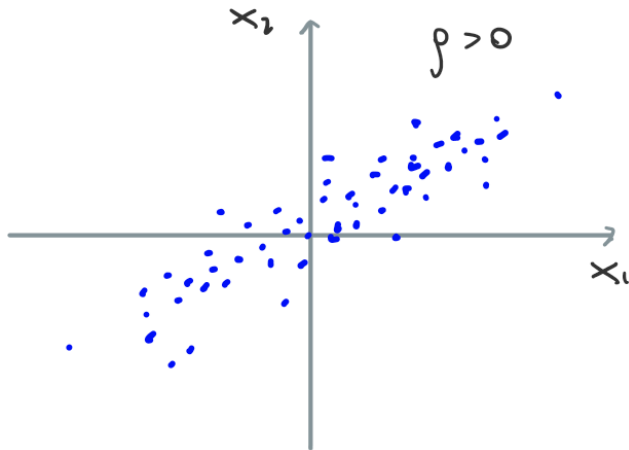
Now, in terms of **variance** we can visually see that feature 1 has higher variance than feature 2, so we could think of select the first feature and remove the other one to predict the points

The idea is that the majority of the information is held by feature 1



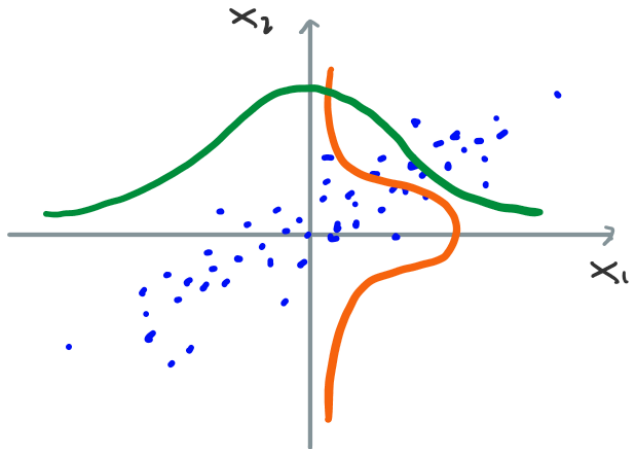
PCA

Independent variables in practice are pretty rare, often times they are **correlated**, so let's here at a case in which the linear correlation coefficient ρ is pretty high



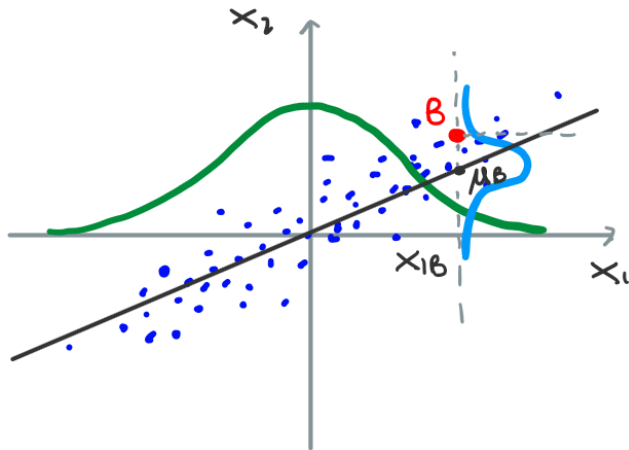
PCA

You can still think of the two separate feature distribution, but just by themselves,
not when they interact



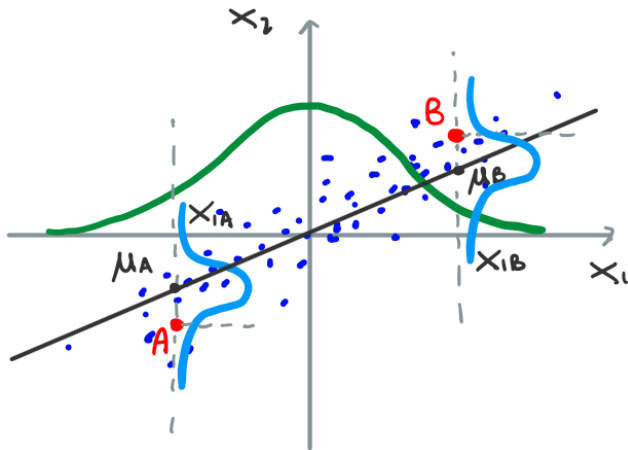
PCA

Idea: If you know already the value of the first feature, the distribution of the second one changes a lot



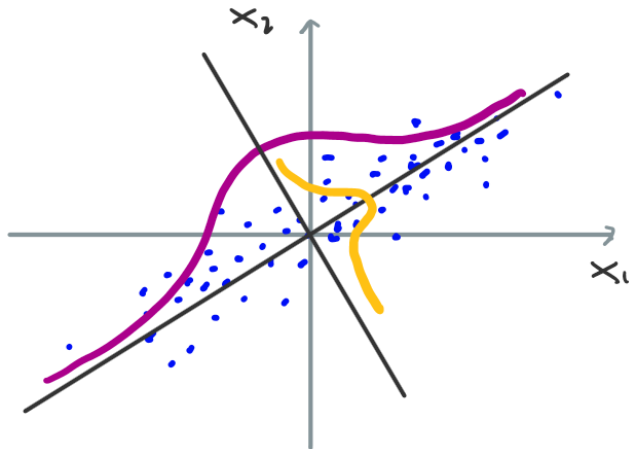
PCA

There's a clear difference between how feature 2 will be, if you know feature 1 is high or low



PCA

Here's the **key idea**: what if we could extract a distribution along the line (purple) so that it contains the MOST amount of **information** (variance) possible?



This is the main idea behind Principal Component Analysis, finding the combination of variables that (alone) contain the highest amount of **variance**

We can then select a few of these "**Principal Components**" so that we reduce the number of features, but keep most of the variance of the whole dataset

Remember: Variance is just a property of a matrix! It's not something bad or something good, it just exists, and if we don't take it into account we lose information on the data!



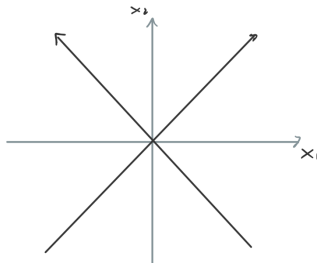
Change of basis

Let's start with the standard basis in $\mathbb{R}^2$:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Now suppose we want to describe things using a new set of axes:

$$\mathbf{b}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{b}_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$



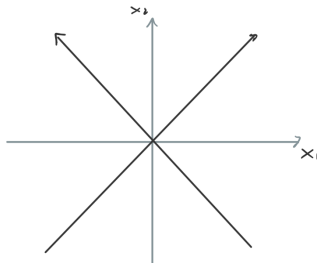
Change of basis

We collect them into a matrix:

$$\mathbf{B} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

If we have a vector $\mathbf{x}$ in the standard basis and we want its coordinates in the new basis, we solve

$$\mathbf{B} \mathbf{x}' = \mathbf{x} \implies \mathbf{x}' = \mathbf{B}^{-1} \mathbf{x}$$



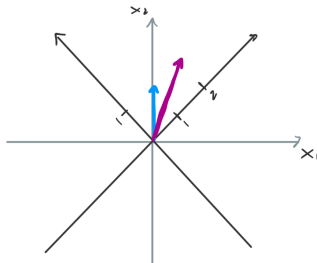
Change of basis

Now suppose we apply a nice, simple transformation in the new basis — for example, a scaling by 2:

$$\mathbf{A}' = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

In the standard basis, the same transformation looks like:

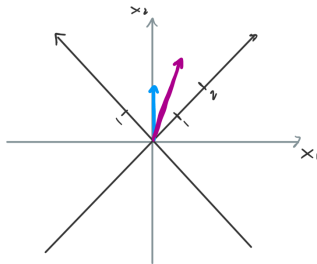
$$\mathbf{A} = \mathbf{B} \mathbf{A}' \mathbf{B}^{-1}$$



Change of basis

So even though $\mathbf{A}'$ was “simple” in the new coordinates, $\mathbf{A}$ can look quite different in the original ones — the same geometric transformation, just seen from a new point of view.

$$\mathbf{A} = \begin{bmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{bmatrix}$$



Back to PCA, remember, we want to **maximize variance**

Given a dataset $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ with zero mean, define the **covariance matrix**:

$$\mathbf{S} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top$$

We want to find a unit vector $\mathbf{w}_1$ that maximizes the variance of the projected data:

$$\max_{\mathbf{w}_1} \mathbf{w}_1^\top \mathbf{S} \mathbf{w}_1 \quad \text{subject to} \quad \|\mathbf{w}_1\| = 1$$



This is the same as minimizing the negative variance (a nice way to make it sound more like optimization!):

$$\min_{\mathbf{w}_1} -\mathbf{w}_1^\top \mathbf{S} \mathbf{w}_1 \quad \text{subject to} \quad \|\mathbf{w}_1\| = 1$$

The solution $\mathbf{w}_1$ is the **eigenvector** of $\mathbf{S}$ corresponding to the largest eigenvalue λ_1 .



Once we find the first principal component $\mathbf{w}_1$, we can project the data onto it and remove that direction of variance.

To find the next one, we solve the same problem again — but this time we add the constraint that the new component must be orthogonal to the previous ones:

$$\max_{\mathbf{w}_k} \mathbf{w}_k^\top \mathbf{S} \mathbf{w}_k \quad \text{subject to} \quad \|\mathbf{w}_k\| = 1, \quad \mathbf{w}_k^\top \mathbf{w}_j = 0 \text{ for all } j < k$$

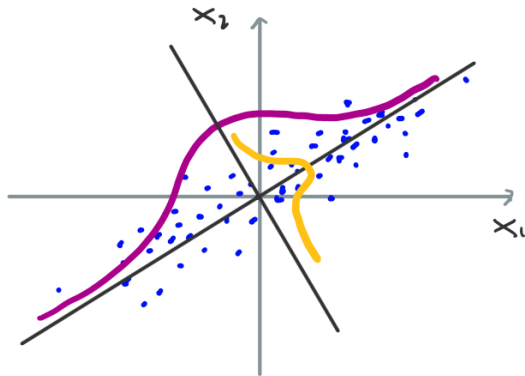
This gives us $\mathbf{w}_2, \mathbf{w}_3, \dots, \mathbf{w}_p$, the remaining eigenvectors of $\mathbf{S}$.

The corresponding eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$ tell us how much variance each component explains.



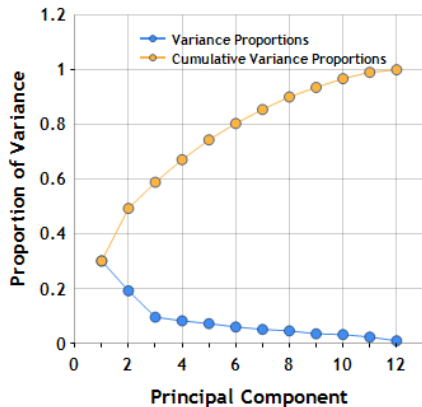
PCA

So, PCA = **find the directions (eigenvectors) that explain the most variance in your data**, aka these axis



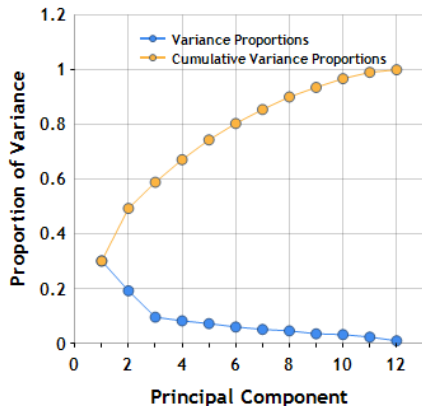
PCA

you can also plot the percentage of variance each component keeps, and plot the cumulative variance



PCA

You can now perform feature selection on these, by keeping the components that maintain 80% or more of the variance!



Important: PCA is designed to maintain variance as information, not distance between data points! So for different scopes we are going to see now, it's not the best choice



Limitations of linear methods

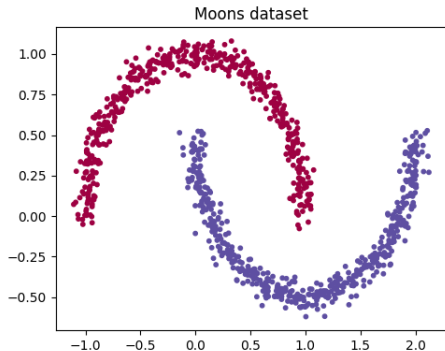
PCA on non-linear data



Where does PCA fail?

PCA attempts to draw lines in the principal uncorrelated directions of your data, only using **linear transformations**. (Remember, it's just a basis change.)

PCA explicitly assumes that the correlation between your features is linear.



What happens if the principal directions can not be described linearly?

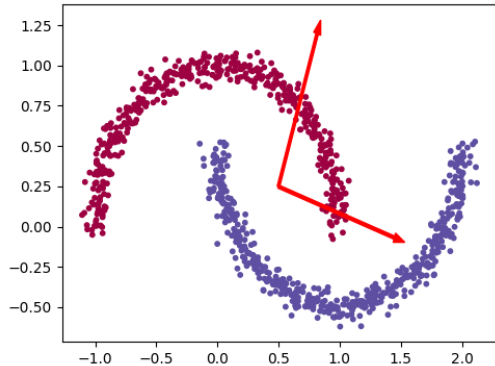
For example, in the two moons dataset, there is no linear vector/straight line separating the given clusters.

Reminder: our goal is to visualise distance between high-dimensional clusters using a reduced number of dimensions.

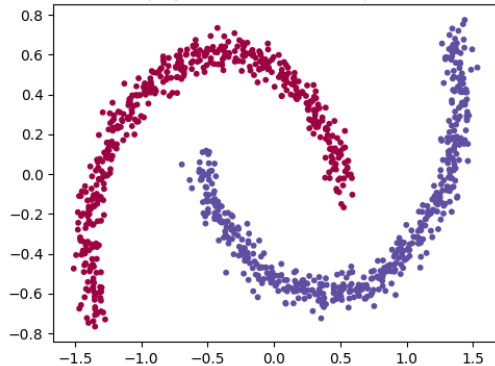


PCA on moons

Moons dataset with PCA directions



PCA projection (fails to unwrap moons)



Limitations of linear methods

Kernel PCA



Kernel methods

- **Intuition:** If we have N data points, they cannot in general be linearly separated in $d < N$ dimensions. However, they can almost always be linearly separated if $d \geq N$.
- **Idea:** Map these N points into a suitable space via $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^N$, where it is easy to draw the line, and do regular PCA there.
- **Issue:** For large N , computing $\phi(x_i)$ may be impossible.
- **Solution:** Just don't compute it then.

It turns out, we can design an algorithm such that we don't ever need to compute the values of ϕ , using kernel functions.



Kernel methods

- **Intuition:** If we have N data points, they cannot in general be linearly separated in $d < N$ dimensions. However, they can almost always be linearly separated if $d \geq N$.
- **Idea:** Map these N points into a suitable space via $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^N$, where it is easy to draw the line, and do regular PCA there.
- **Issue:** For large N , computing $\phi(x_i)$ may be impossible.
- **Solution:** Just don't compute it then.

It turns out, we can design an algorithm such that we don't ever need to compute the values of ϕ , using kernel functions.



Kernel methods

- **Intuition:** If we have N data points, they cannot in general be linearly separated in $d < N$ dimensions. However, they can almost always be linearly separated if $d \geq N$.
- **Idea:** Map these N points into a suitable space via $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^N$, where it is easy to draw the line, and do regular PCA there.
- **Issue:** For large N , computing $\phi(x_i)$ may be impossible.
- **Solution:** Just don't compute it then.

It turns out, we can design an algorithm such that we don't ever need to compute the values of ϕ , using kernel functions.



Kernel methods

- **Intuition:** If we have N data points, they cannot in general be linearly separated in $d < N$ dimensions. However, they can almost always be linearly separated if $d \geq N$.
- **Idea:** Map these N points into a suitable space via $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^N$, where it is easy to draw the line, and do regular PCA there.
- **Issue:** For large N , computing $\phi(x_i)$ may be impossible.
- **Solution:** Just don't compute it then.

It turns out, we can design an algorithm such that we don't ever need to compute the values of ϕ , using kernel functions.



The Kernel trick

A **kernel function** is a symmetric function $k : X \times X \rightarrow \mathbb{R}$ that can be written as an inner product in some other vector space Y , i.e. $k(x, y) = \langle \phi(x), \phi(y) \rangle_Y$. So by constructing PCA via inner products, we can use k and avoid computing ϕ .

Instead of working with $X = [x_1, \dots, x_N]^T$, we work with $\Phi = [\phi(x_1), \dots, \phi(x_n)]^T$, in a space of high enough dimension that $\phi(x_i)$ is a basis. Then PCA is simply solving for eigenvectors of $\frac{1}{n}\Phi^T\Phi$, and writing each eigenvector in the basis of $\phi(x_i)$ we get:

$$\frac{1}{n}\Phi^T\Phi\Phi^T\alpha = \lambda\Phi^T\alpha \implies K\alpha = n\lambda\alpha$$

where $K_{ij} = k(x_i, x_j)$. So we only require finding eigenvectors α_i of K .

Once we have these eigenvectors, we can return to X via the inner product and k .



The Kernel trick

A **kernel function** is a symmetric function $k : X \times X \rightarrow \mathbb{R}$ that can be written as an inner product in some other vector space Y , i.e. $k(x, y) = \langle \phi(x), \phi(y) \rangle_Y$. So by constructing PCA via inner products, we can use k and avoid computing ϕ .

Instead of working with $X = [x_1, \dots, x_N]^T$, we work with $\Phi = [\phi(x_1), \dots, \phi(x_n)]^T$, in a space of high enough dimension that $\phi(x_i)$ is a basis. Then PCA is simply solving for eigenvectors of $\frac{1}{n}\Phi^T\Phi$, and writing each eigenvector in the basis of $\phi(x_i)$ we get:

$$\frac{1}{n}\Phi^T\Phi\Phi^T\alpha = \lambda\Phi^T\alpha \implies K\alpha = n\lambda\alpha$$

where $K_{ij} = k(x_i, x_j)$. So we only require finding eigenvectors α_i of K .

Once we have these eigenvectors, we can return to X via the inner product and k .



Common kernels

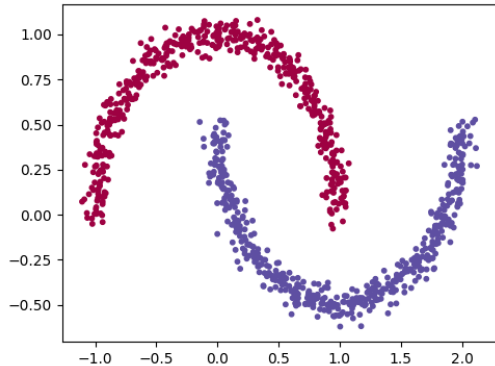
There is a lot of theory on something called *Reproducing Kernel Hilbert Spaces*, which discusses kernel functions, nice properties, and how to construct good functions. In practice, you will always use one of these kernels:

Kernel	Formula $k(x, y)$	Intuition
Linear	$x^T y$	No change, still linear
Polynomial	$(x^T y + c)^p$	All polynomial curves
RBF/Gaussian	$\exp(-\ x - y\ ^2 / 2\sigma^2)$	Infinite-dimensional, combination of all bump functions at each data point
Sigmoid	$\tanh(\alpha x^T y + c)$	Roughly, attempts to mimics neural network activation behaviour

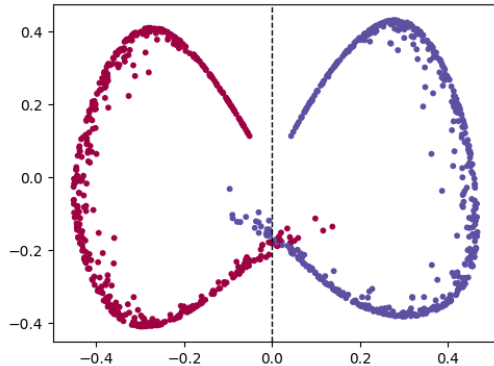


Kernels on moons

Moons dataset

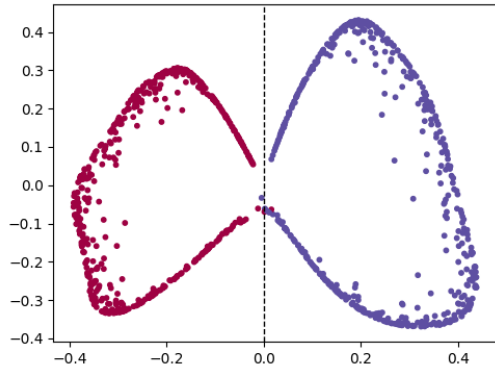


Kernel PCA (RBF kernel) succeeds on moons

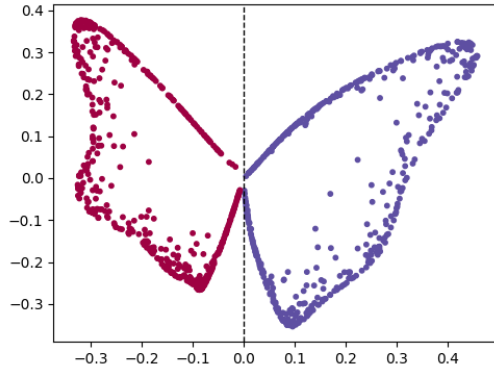


Kernels on moons

Kernel PCA (RBF kernel) succeeds on moons

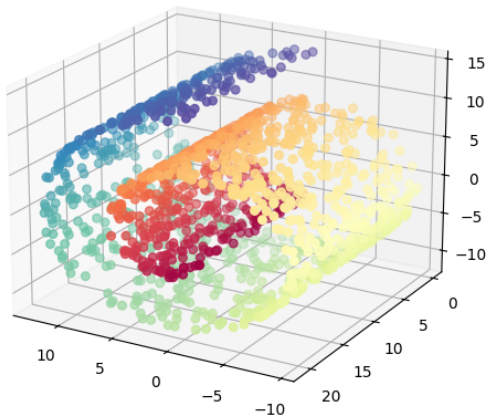


Kernel PCA (RBF kernel) succeeds on moons



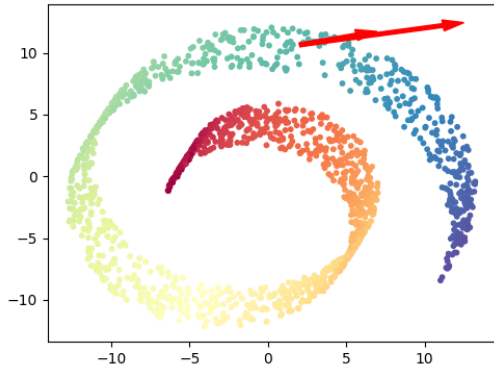
Kernels on Swiss rolls

Swiss roll

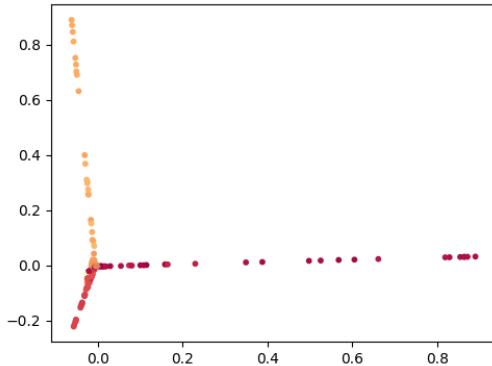


Kernels on Swiss rolls

PCA on Swiss roll

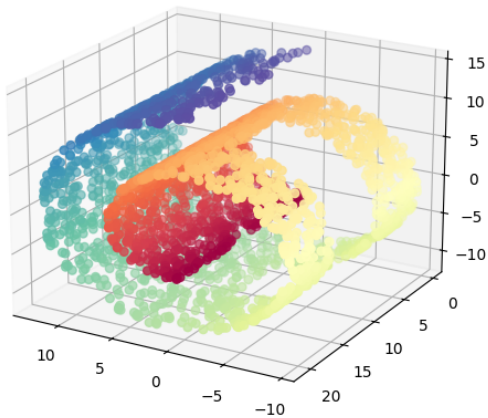


Kernel PCA on Swiss roll



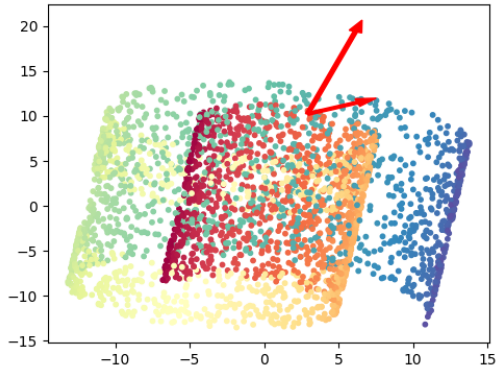
Kernels on Swiss rolls

Swiss roll with hole

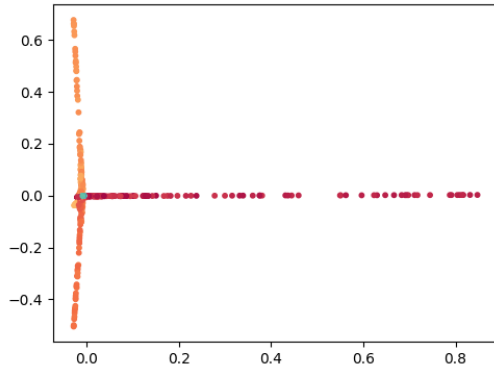


Kernels on Swiss rolls

PCA on Swiss roll with hole



Kernel PCA on Swiss roll with hole



Non-linear dimensionality reduction

A paradigm shift



Non-linear dimensions

We need something new. Clearly in the Swiss roll example, we have a simple 2d rectangle, which has been rolled into a 3d space. PCA could easily explain this 2D space when unrolled.

Intuitively, we know that nothing has really changed about this 2d surface when we bend it in 3d: it is still inherently a 2-dimensional object, we're just viewing it differently, but PCA can't see this.

So what do we actually need?

- we want to draw lines;
- we want to calculate gradients;
- and we want some notion of the "real" dimension of the object, regardless of the space it lies in.



Non-linear dimensions

We need something new. Clearly in the Swiss roll example, we have a simple 2d rectangle, which has been rolled into a 3d space. PCA could easily explain this 2D space when unrolled.

Intuitively, we know that nothing has really changed about this 2d surface when we bend it in 3d: it is still inherently a 2-dimensional object, we're just viewing it differently, but PCA can't see this.

So what do we actually need?

- we want to draw lines;
- we want to calculate gradients;
- and we want some notion of the "real" dimension of the object, regardless of the space it lies in.



Manifolds

It turns out that we already have a mathematical object which does exactly this. A smooth **manifold** is a topological space that locally resembles Euclidean space near each point, and in these local neighbourhoods we can differentiate.

What does this give us?

- Every point has a neighbourhood which is essentially a small piece of $\mathbb{R}^d$ for some d ;
- and in these neighbourhoods, we can differentiate as normal for $\mathbb{R}^d$.
- Finally, the dimension of the manifold is defined to be exactly d .

So if we can model surfaces such as the Swiss roll as a 2d manifold in 3d space, we immediately have dimension reduction. So the goal is to learn a **latent manifold** for our data, where similar objects are close on the manifold.¹

¹This is why nonlinear dimensionality reduction is also known as manifold learning.



Manifolds

It turns out that we already have a mathematical object which does exactly this. A smooth **manifold** is a topological space that locally resembles Euclidean space near each point, and in these local neighbourhoods we can differentiate.

What does this give us?

- Every point has a neighbourhood which is essentially a small piece of $\mathbb{R}^d$ for some d ;
- and in these neighbourhoods, we can differentiate as normal for $\mathbb{R}^d$.
- Finally, the dimension of the manifold is defined to be exactly d .

So if we can model surfaces such as the Swiss roll as a 2d manifold in 3d space, we immediately have dimension reduction. So the goal is to learn a **latent manifold** for our data, where similar objects are close on the manifold.¹

¹This is why nonlinear dimensionality reduction is also known as manifold learning.



Manifolds

It turns out that we already have a mathematical object which does exactly this. A smooth **manifold** is a topological space that locally resembles Euclidean space near each point, and in these local neighbourhoods we can differentiate.

What does this give us?

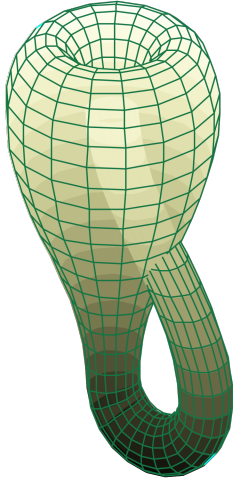
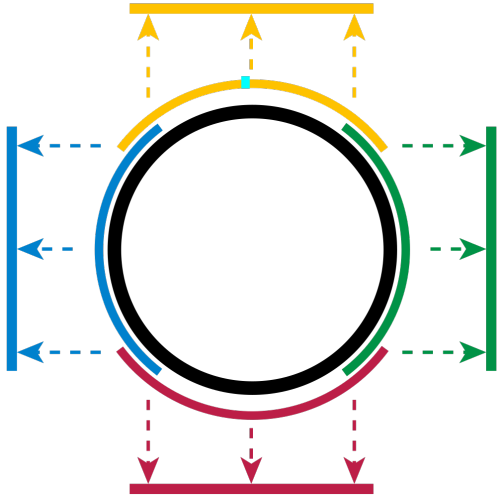
- Every point has a neighbourhood which is essentially a small piece of $\mathbb{R}^d$ for some d ;
- and in these neighbourhoods, we can differentiate as normal for $\mathbb{R}^d$.
- Finally, the dimension of the manifold is defined to be exactly d .

So if we can model surfaces such as the Swiss roll as a 2d manifold in 3d space, we immediately have dimension reduction. So the goal is to learn a **latent manifold** for our data, where similar objects are close on the manifold.¹

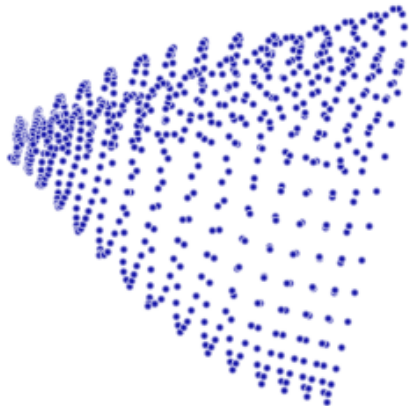
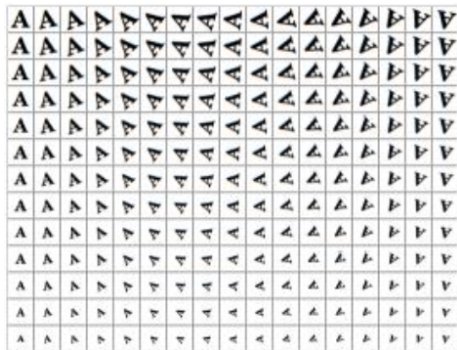
¹This is why nonlinear dimensionality reduction is also known as manifold learning.



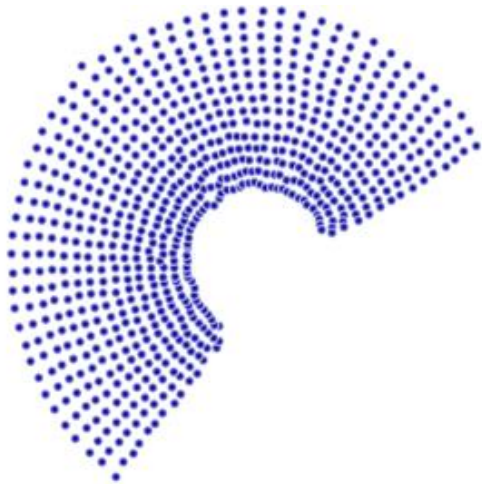
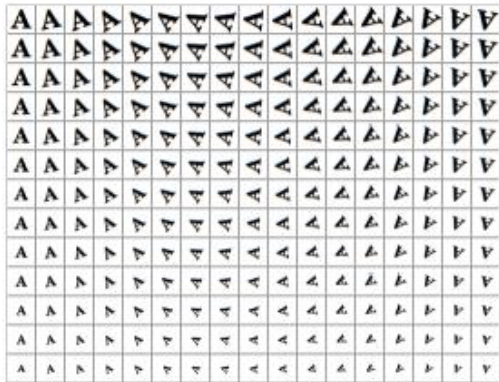
Manifolds: some examples



Letter rotation: PCA



Letter rotation: Manifolds



▼ Important concepts

- Sammon's mapping
- Self-organizing map
- Kernel principal component analysis
- Principal curves and manifolds
- Laplacian eigenmaps
- Isomap
- Locally-linear embedding
 - Hessian locally-linear embedding (Hessian LLE)
 - Modified Locally-Linear Embedding (MLLE)
- Local tangent space alignment
- Maximum variance unfolding
- Autoencoders
- Gaussian process latent variable models
- t-distributed stochastic neighbor embedding

▼ Other algorithms

- Relational perspective map
- Contagion maps
- Curvilinear component analysis
- Curvilinear distance analysis
- Diffeomorphic dimensionality reduction
- Manifold alignment
- Diffusion maps
- Local multidimensional scaling
- Nonlinear PCA
- Data-driven high-dimensional scaling
- Manifold sculpting
- RankVisu
- Topologically constrained isometric embedding
- Uniform manifold approximation and projection

How do we do this?

What methods/algorithms are available?

Well, there's a lot. Here are just the ones listed on the wikipedia page, and there's new research papers on these methods quite regularly.

Let's just look at two or three important ones to try and get the concept.



Non-linear dimensionality reduction

t-SNE



t-distributed Stochastic Neighbour Embedding

Goal: model high-dimensional data in 2d or 3d for cluster visualisation.

Ideology: Preserve pairwise similarity: close high-dim points should be close low-dim points.

Let $X = \{x_1, \dots, x_N\} \subseteq \mathbb{R}^d$ for some large d . Then we define $w(x_i, x_j) = \exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)$. Note that this is the Gaussian kernel from earlier, and behaves like a similarity metric².

We then define conditional probabilities: $p_{j|i} = w(x_i, x_j) / \sum_{k \neq i} w(x_i, x_k)$.

Intuition: $p_{j|i}$ measures how likely x_i would pick x_j as its nearest neighbour.

Then symmetrise for a joint probability: $p_{ij} = (p_{j|i} + p_{i|j}) / 2N$. Then p_{ij} should be large for close points, and small for far points.

We have now established a measure for similarity in the high-dimensional space.

² σ_i here is a training parameter, controlling number of neighbours



t-distributed Stochastic Neighbour Embedding

Goal: model high-dimensional data in 2d or 3d for cluster visualisation.

Ideology: Preserve pairwise similarity: close high-dim points should be close low-dim points.

Let $X = \{x_1, \dots, x_N\} \subseteq \mathbb{R}^d$ for some large d . Then we define $w(x_i, x_j) = \exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)$. Note that this is the Gaussian kernel from earlier, and behaves like a similarity metric².

We then define conditional probabilities: $p_{j|i} = w(x_i, x_j) / \sum_{k \neq i} w(x_i, x_k)$.

Intuition: $p_{j|i}$ measures how likely x_i would pick x_j as its nearest neighbour.

Then symmetrise for a joint probability: $p_{ij} = (p_{j|i} + p_{i|j}) / 2N$. Then p_{ij} should be large for close points, and small for far points.

We have now established a measure for similarity in the high-dimensional space.

² σ_i here is a training parameter, controlling number of neighbours



t-distributed Stochastic Neighbour Embedding

Goal: model high-dimensional data in 2d or 3d for cluster visualisation.

Ideology: Preserve pairwise similarity: close high-dim points should be close low-dim points.

Let $X = \{x_1, \dots, x_N\} \subseteq \mathbb{R}^d$ for some large d . Then we define $w(x_i, x_j) = \exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)$. Note that this is the Gaussian kernel from earlier, and behaves like a similarity metric².

We then define conditional probabilities: $p_{j|i} = w(x_i, x_j) / \sum_{k \neq i} w(x_i, x_k)$.

Intuition: $p_{j|i}$ measures how likely x_i would pick x_j as its nearest neighbour.

Then symmetrise for a joint probability: $p_{ij} = (p_{j|i} + p_{i|j}) / 2N$. Then p_{ij} should be large for close points, and small for far points.

We have now established a measure for similarity in the high-dimensional space.

² σ_i here is a training parameter, controlling number of neighbours



t-SNE: low-dimensional similarity

To construct low-dimensional points that maintain pairwise similarity, we compare with a low-dim similarity metric. Define:

$$w(y_i, y_j) = \frac{1}{1 + \|y_i - y_j\|^2}, \implies q_{ij} = \frac{w(y_i, y_j)}{\sum_{k \neq l} w(y_k, y_l)}.$$

This is the exact same probability setup as before, but using the t -distribution instead of the Gaussian. Why?

The Gaussian decays very rapidly. The original idea used the Gaussian in both spaces, but it was found that this quickly led to collapsing clusters to single points in the low-dimensional space. The t -distribution decays more moderately, leading to better visualisation.



t-SNE: low-dimensional similarity

To construct low-dimensional points that maintain pairwise similarity, we compare with a low-dim similarity metric. Define:

$$w(y_i, y_j) = \frac{1}{1 + \|y_i - y_j\|^2}, \implies q_{ij} = \frac{w(y_i, y_j)}{\sum_{k \neq l} w(y_k, y_l)}.$$

This is the exact same probability setup as before, but using the t -distribution instead of the Gaussian. Why?

The Gaussian decays very rapidly. The original idea used the Gaussian in both spaces, but it was found that this quickly led to collapsing clusters to single points in the low-dimensional space. The t -distribution decays more moderately, leading to better visualisation.



But what actually are the points y_i ? We find them via gradient descent on the KL divergence:

$$KL(P||Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$

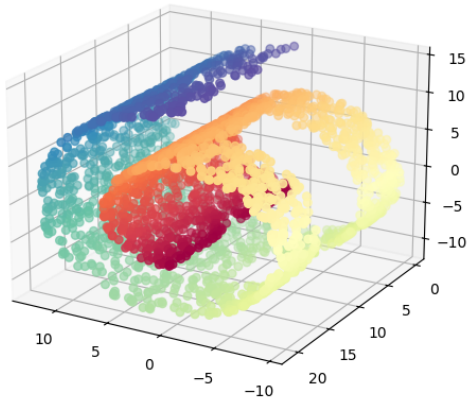
This measures the divergence between the probability distributions. Intuitively, if $p_{ij} > q_{ij}$, we push points closer together in the low-dim space. Note that we can often start with random initialisation of the points y_i .

Let's try the Swiss roll again.

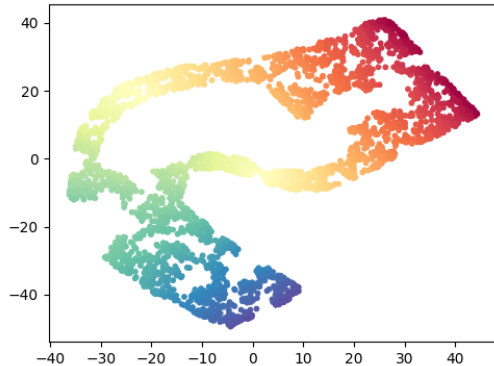


t-SNE on Swiss rolls

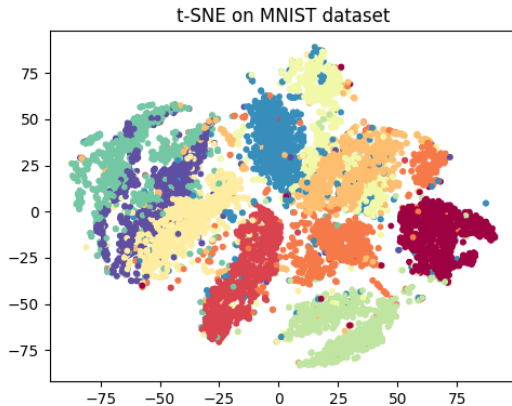
Swiss roll with hole



t-SNE on Swiss roll with hole

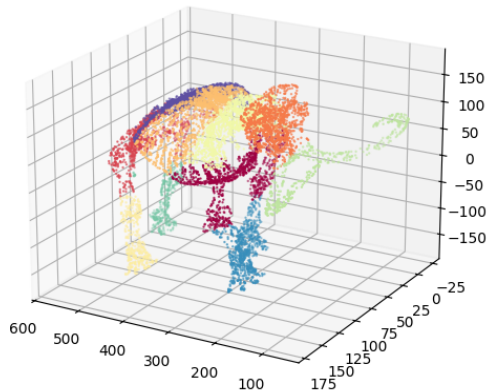


t-SNE on MNIST: 784 to 2

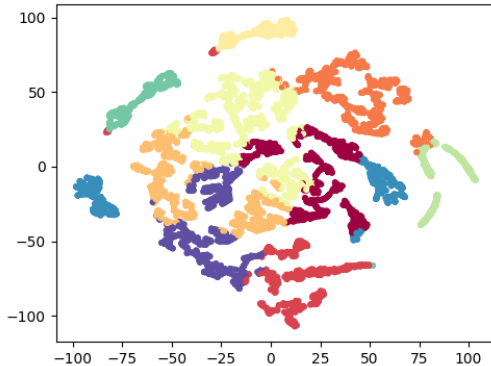


t-SNE on Mammoth: global distortion

3D Mammoth scan



t-SNE on Mammoth dataset



Non-linear dimensionality reduction

UMAP



Uniform Manifold Approximation and Projection

UMAP fixes several flaws with t-SNE. t-SNE focuses heavily on pairwise similarity in local neighbourhoods. This topological approach leads to a stretching with abandons all global structure. Also, the pairwise sum is computationally expensive.

Goal: adapt t-SNE's ideas to do the same thing with a little more global structure.

Why? because the goal is intuition of clusters in a global dataset. t-SNE can visualise clusters, but not how far away they are from other clusters globally.

Idea: restrict the sum to nearest neighbours, and use the distance metric on the manifold.



Uniform Manifold Approximation and Projection

UMAP fixes several flaws with t-SNE. t-SNE focuses heavily on pairwise similarity in local neighbourhoods. This topological approach leads to a stretching with abandons all global structure. Also, the pairwise sum is computationally expensive.

Goal: adapt t-SNE's ideas to do the same thing with a little more global structure.

Why? because the goal is intuition of clusters in a global dataset. t-SNE can visualise clusters, but not how far away they are from other clusters globally.

Idea: restrict the sum to nearest neighbours, and use the distance metric on the manifold.



Uniform Manifold Approximation and Projection

UMAP fixes several flaws with t-SNE. t-SNE focuses heavily on pairwise similarity in local neighbourhoods. This topological approach leads to a stretching with abandons all global structure. Also, the pairwise sum is computationally expensive.

Goal: adapt t-SNE's ideas to do the same thing with a little more global structure.

Why? because the goal is intuition of clusters in a global dataset. t-SNE can visualise clusters, but not how far away they are from other clusters globally.

Idea: restrict the sum to nearest neighbours, and use the distance metric on the manifold.



UMAP: high-dimensional graph

Again, say we have points x_i in some high-dim space. Choose some k . Then:

- For each x_i , choose its k -nearest neighbour points.
- Define a fuzzy set membership for each neighbour x_j :

$$\mu_{ij} = \exp \left(- \frac{\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i} \right)$$

where ρ_i is the distance to the nearest neighbour, and σ_i is a training parameter like in t-SNE.

- Symmetrise the fuzzy membership (fuzzy set union): $\tilde{\mu}_{ij} = \mu_{ij} + \mu_{ji} - \mu_{ij}\mu_{ji}$.

This constructs a graph where each point is connected to its k -nearest neighbours with edge weight representing closeness in the high-dimensional space. We can then sum over edges instead of pairwise.



UMAP: low-dim and optimisation

We then define the low-dimensional similarity by $v_{ij} = 1/(1 + a \|y_i - y_j\|^{2b})$ for parameters a, b . This is a parameterised version of the t -distribution before, for the same reasons.

We then optimise over cross-entropy loss using stochastic gradient descent:

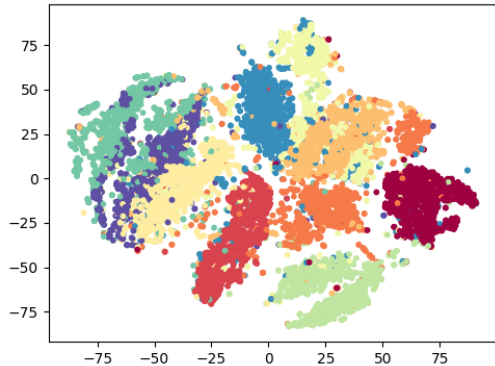
$$CE = \sum_{i < j} \tilde{\mu}_{ij} \log v_{ij} + (1 - \tilde{\mu}_{ij}) \log(1 - v_{ij})$$

Result: the graph nature preserves global structure better, and summing over less pairs speeds up the process significantly.

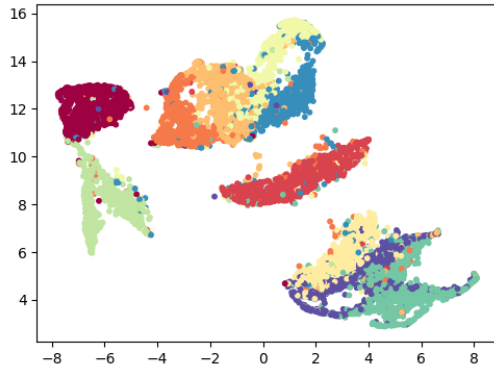


UMAP on MNIST: more global separation

t-SNE on MNIST dataset

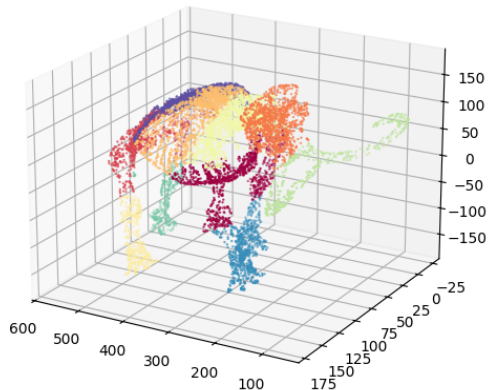


UMAP on MNIST dataset

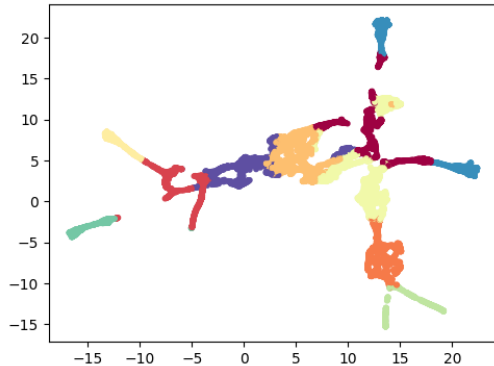


UMAP on Mammoth: more global separation

3D Mammoth scan

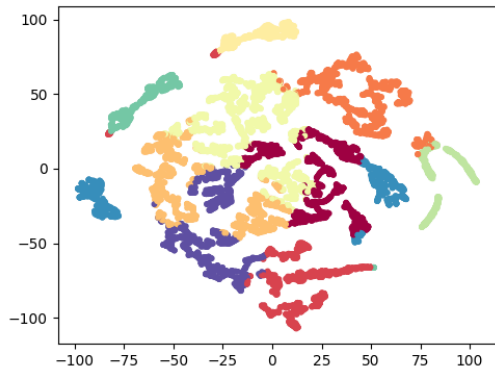


UMAP on Mammoth dataset

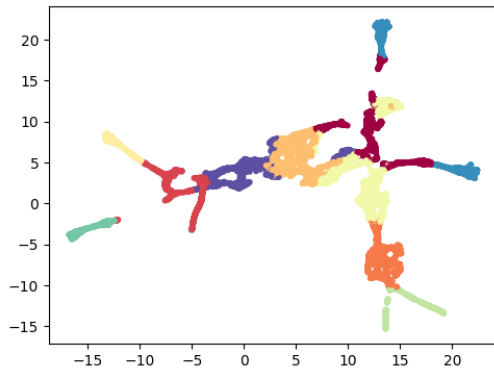


UMAP on Mammoth: more global separation

t-SNE on Mammoth dataset



UMAP on Mammoth dataset



Non-linear dimensionality reduction

PaCMAP



PacMAP: Pairwise Controlled Manifold Approximation

PaCMAP is a modern adaption of UMAP and some other similar dimension reduction tools with very impressive performance.

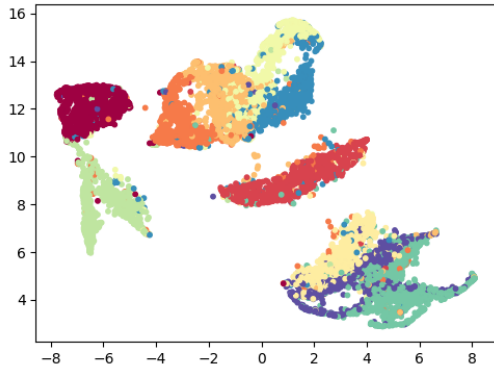
- The authors released a 73 page paper which I have not had time to read yet, but I wanted to show you the performance.
- The paper shows that in many cases, PaCMAP with default parameters outperforms tuned UMAP, and does it faster.
- The key idea was to split the loss function sum into near, medium, and far pairs. The loss function then attracts near points, repels far points, and maintains the distance of medium points.

From my testing on the mammoth set: PaCMAP was **significantly** faster than both t-SNE and UMAP.

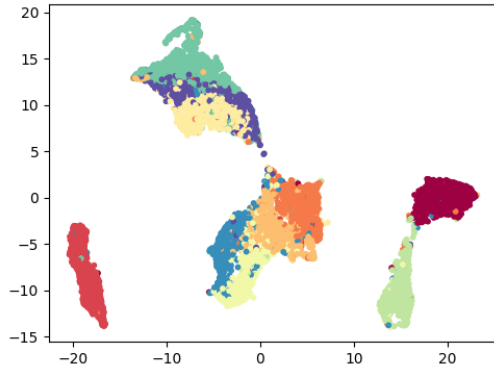


PaCMAP on MNIST

UMAP on MNIST dataset

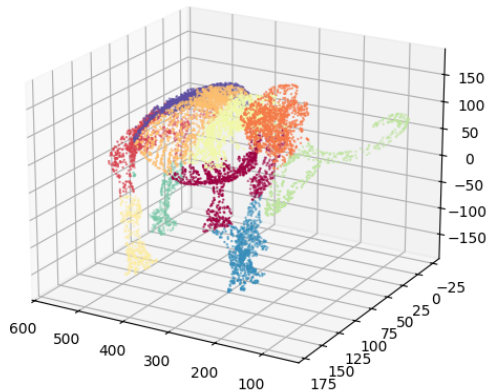


PaCMAP on MNIST dataset

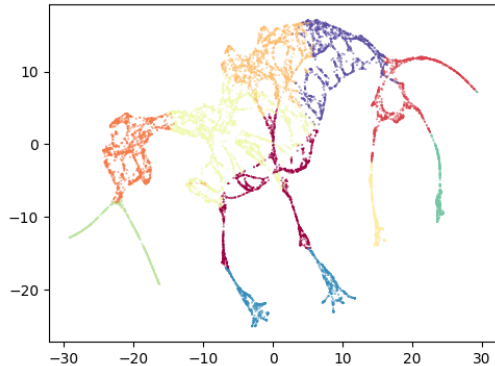


PaCMAP on Mammoth

3D Mammoth scan

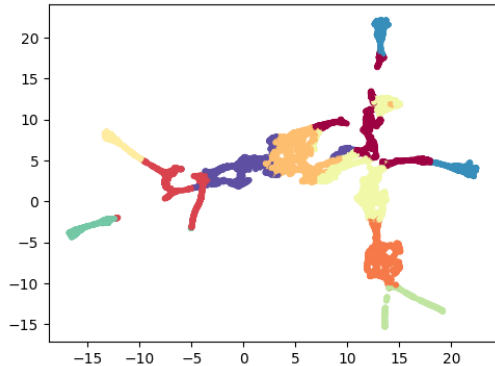


PaCMAP on Mammoth dataset

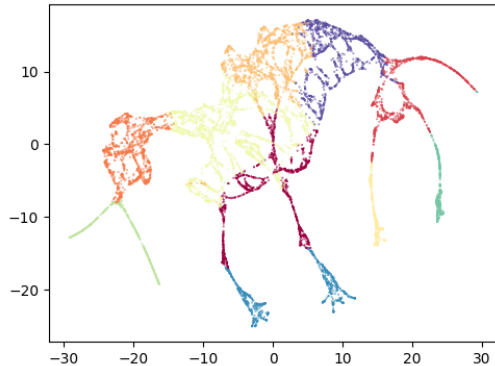


PaCMAP on Mammoth

UMAP on Mammoth dataset



PaCMAP on Mammoth dataset



Summary

- k-means is a simple algorithm for clustering unlabelled similar points.
- Dimension reduction is a key tool for visualising clusters in high-dimensions.
- Kernels allow us to adapt linear methods to basic non-linear problems.
- Manifolds allow us to simplify learning on more complex curved surfaces.

Please fill out the feedback form!

